

Timesheet & ITSM Agentic Platform

A two-part system built on one Next.js app + Supabase, deployed on Railway, and driven from **Slack via Salesforce Agentforce agents**:

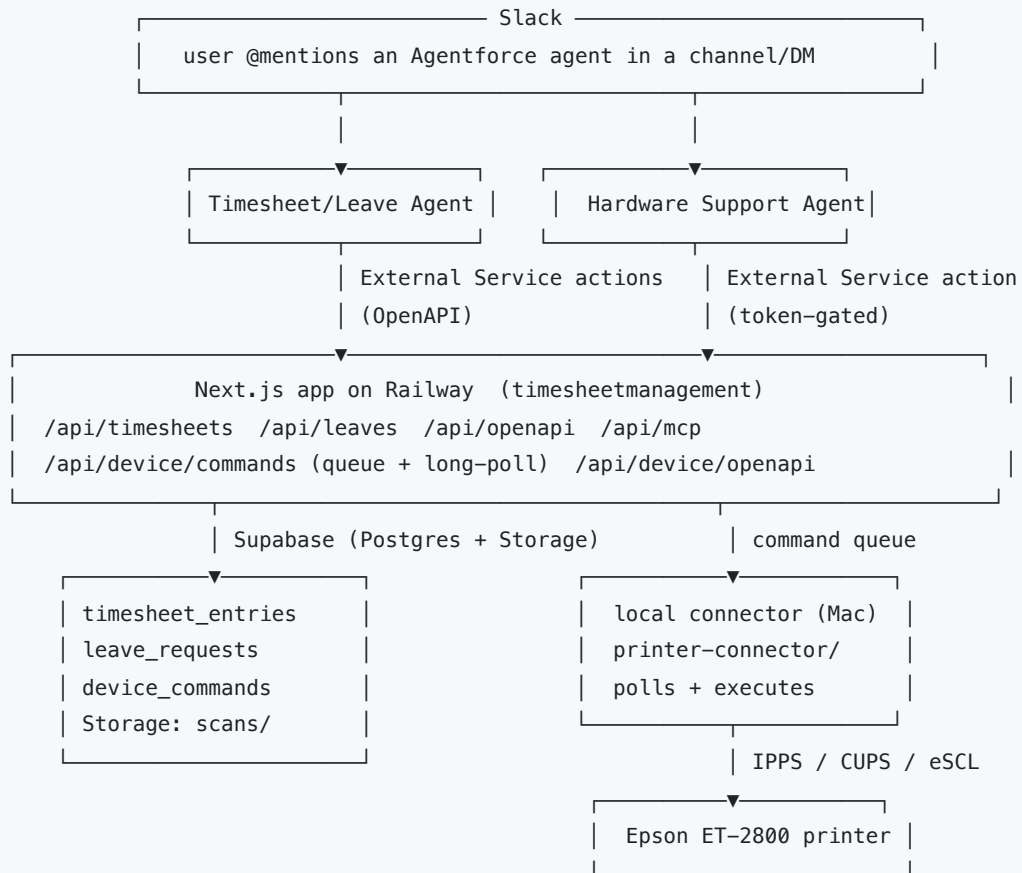
1. **Timesheet & Leave Portal** — log daily hours and apply for leave from a web UI, from Claude (MCP), or from Salesforce/Slack (External Services + an agent that can even read a project channel's week of conversation and log it).
2. **ITSM Hardware Agent POC** — a Slack agent that **remotely operates a physical Epson ET-2800 printer** (diagnose, print, identify, clear queue, scan) through a cloud relay + a local connector.

Users are identified by **email**. No part of this requires the browser to touch the database directly — everything goes through the app's API.

Table of contents

- [Architecture](#)
 - [Tech stack](#)
 - [Part 1 — Timesheet & Leave Portal](#)
 - [Part 2 — ITSM Hardware Agent POC](#)
 - [Deployment & configuration](#)
 - [Key learnings / gotchas](#)
 - [Roadmap](#)
-

Architecture



Also reachable directly: the **web UI** (/), **MCP** clients (/api/mcp), and any REST client.

Tech stack

- **Next.js 16** (App Router, TypeScript) + Tailwind — web UI + all API routes
- **Supabase** (Postgres + Storage) — data + scanned-image hosting
- **Railway** — hosting (Nixpacks, Node 22)
- **Model Context Protocol** — hand-rolled JSON-RPC endpoint
- **Salesforce** — External Services (OpenAPI), Agentforce agents, Apex
- **Slack** — Agentforce in Slack
- **Local connector** — Node service talking IPPS/CUPS/eSCL to the printer

Part 1 — Timesheet & Leave Portal

Identity

Everyone is identified by **email**. The web UI stores it in `localStorage` ; MCP and Salesforce actions pass it explicitly.

Web UI (/)

- **Timesheet tab** — a week navigator with all 7 days in a row; add hours + description **per day**; per-day totals (green $\geq 8h$, orange if $< 8h$); inline delete; "View all timesheets" history grouped by week.
- **Leave tab** — apply for leave (date range, type, reason), see requests with color-coded status, and cancel them.
- **Leave-day block** — you can't log hours on a day you're on leave (enforced server-side, so it holds for every channel).

Three interfaces, one logic

All surfaces share `src/lib/timesheet.ts` / `src/lib/leave.ts` , so validation, week normalization (a week = its **Monday**), and behavior are identical.

Interface	Where
Web UI	<code>/</code>
MCP	<code>/api/mcp</code>
REST + OpenAPI	<code>/api/timesheets</code> , <code>/api/leaves</code> , spec at <code>/api/openapi</code>

MCP tools

`log_timesheet` (email, hours, description, datelweek) · `list_timesheets` · `apply_leave` · `list_leaves` .

REST API

Method	Path	Body / query
GET	<code>/api/timesheets</code>	<code>?email=&week=</code>
POST	<code>/api/timesheets</code>	<code>{ email, hours, description, date? week? }</code>
DELETE	<code>/api/timesheets/{id}</code>	<code>?email=</code>
GET	<code>/api/leaves</code>	<code>?email=</code>
POST	<code>/api/leaves</code>	<code>{ email, start_date, end_date?, leave_type?, reason? }</code>
DELETE	<code>/api/leaves/{id}</code>	<code>?email=</code>

Salesforce External Service actions (from `/api/openapi`)

`logTimesheet` , `listTimesheets` , `deleteTimesheet` , `applyLeave` , `listLeaves` , `cancelLeave` — registered via a no-auth Named Credential.

Agentforce + Slack

A Timesheet/Leave agent in Slack uses the above actions. It can also **read a project channel's conversation for a week, summarize per day, and log timesheets** — via a custom Apex action (`SlackChannelReader`) that calls Slack's `conversations.history` (the standard "Search" action needs Enterprise Grid + Slack AI, so Apex is used instead). Leave applications also post a notification to a Slack channel through a Flow.

Database

```
create table public.timesheet_entries (
  id uuid primary key default gen_random_uuid(),
  user_email text not null,
  week_start date not null,    -- Monday of the week
  work_date date,             -- the specific day worked
  description text not null,
  hours numeric(5,2) not null check (hours >= 0 and hours <= 168),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.leave_requests (
  id uuid primary key default gen_random_uuid(),
  user_email text not null,
  start_date date not null,
  end_date date not null,
  leave_type text not null default 'vacation', -- vacation|sick|personal|other
  reason text,
  status text not null default 'pending',      -- pending|approved|rejected|cancelled
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Both have RLS enabled with no policies — service-role-only (server) access.

Part 2 — ITSM Hardware Agent POC

A Slack agent that remotely operates a physical **Epson ET-2800**. The cloud can't reach a printer on a private LAN, so a **local connector** bridges them.

Flow

```
Slack → Hardware Support Agent → runPrinterCommand (External Service, token-gated)
  → POST /api/device/commands (enqueue + long-poll up to 55s)
  → device_commands queue (Supabase)
  → local connector polls /api/device/commands/next, executes on the printer,
    posts the result to /api/device/commands/{id}/result
  → result (incl. a human-readable `summary`) returns to the agent → Slack
```

Polling means **no firewall/tunnel** is needed; the connector reaches out.

Connector commands

Command	Action
<code>status</code>	state, accepting-jobs, fault reasons, ink levels (IPPS)
<code>identify</code>	printer beeps/flashes (IPP Identify-Printer)
<code>print_test</code>	print a diagnostic page
<code>print_text</code>	print provided text
<code>print_file</code>	download a URL and print it
<code>scan</code>	scan the glass (eSCL) and return the image
<code>clear_queue</code>	clear stuck print jobs
<code>restart</code>	stub — needs a smart plug for a real power-cycle

Printer specifics (ET-2800)

- Discovered via Bonjour (`dns-sd -B _ipp._tcp`). IP `10.0.0.3` .
- **Requires TLS** → `ipps://10.0.0.3:631/ipp/print` (plain `ipp` returns HTTP 426). Self-signed cert → connector accepts it (LAN-only).
- Accepts JPEG/URF/PWG-raster/ESC-PR but **not PDF**, so printing goes through **CUPS lp** (driverless `everywhere` queue) which converts.
- **Scan** via eSCL/AirScan at `https://10.0.0.3:443/eSCL` (PDF/JPEG, flatbed): POST `ScanJobs` → GET `NextDocument` → DELETE the job to release it.

Salesforce setup (token-gated)

Separate from the no-auth timesheet service: External Service **PrinterControl** via Named Credential **TimesheetDevice** + External Credential **DeviceApi** (Custom) injecting `Authorization: Bearer <DEVICE_API_TOKEN>` . Generates one action, `runPrinterCommand` , added to a **Hardware Support** agent topic.

Cloud relay endpoints

- `POST /api/device/commands` — enqueue + long-poll (token-gated)

- `GET /api/device/commands/next?device=` — connector claims next command
- `POST /api/device/commands/{id}/result` — connector reports outcome
- `POST /api/device/scan-upload` — stores a scan in the public `scans` bucket

Open issue (scan delivery)

The Einstein **Trust Layer masks URLs** in the agent's reply, so the scanned-image link shows as `URL_Redacted` to the user (the raw action output still has the real URL). The in-progress fix: have the connector post the scanned image **straight into Slack as a file** (Slack `files.uploadV2`) so there's no URL to mask — gated by `SLACK_BOT_TOKEN` + `SLACK_SCAN_CHANNEL` (Slack app/token not yet created).

Deployment & configuration

Railway (the Next.js app)

- Build: Nixpacks, **Node 22** (pinned via `.nvmrc` + `package.json` engines).
- Start: `npm start` (binds Railway's `$PORT` automatically — don't set a port).
- Variables: `NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `DEVICE_API_TOKEN`, optional `APP_BASE_URL`.
- ⚠️ `NEXT_PUBLIC_SUPABASE_URL` is inlined at **build** time — changing the Supabase project requires a **redploy**, not just a variable update.

Supabase

Tables `timesheet_entries`, `leave_requests`, `device_commands`; public Storage bucket `scans`. RLS on (service-role only).

Local connector

`printer-connector/` — run `sh start.sh` (sets `RELAY_URL`, `DEVICE_API_TOKEN`, `PRINTER_IP`, optional Slack vars). Must stay running for the agent to reach the printer.

Key learnings / gotchas

- **Railway Node version:** Nixpacks defaulted to Node 18 (Next 16 needs ≥ 20.9); pin via `.nvmrc` + `engines` (the `railway.toml` key is ignored).
- **NEXT_PUBLIC vars are build-time inlined** — symptom of a stale value: "Invalid API key" when the key is right but the URL is from an old build.
- **Salesforce External Services + Apex reserved words:** an OpenAPI field named `date` was mangled to `z0date`; renamed the input to `work_date`.
- **No-auth vs token-gated Named Credentials:** for the device API, bake `Bearer` into the External Credential auth-parameter value and set the custom header to `{!$Credential.DeviceApi.Token}` (the `'Bearer ' & {!...}` formula failed).

- **Agentforce in Slack:** the agent's **label must contain "Agent"** to appear in Slack admin; a Slack workspace pairs with one Salesforce org; the standard **Search** action needs Enterprise Grid + Slack AI (so channel reads use Apex).
 - **Printer needs TLS** (`ipps`), accepts no PDF (use CUPS), and the **scanner must release its job** or the next scan returns HTTP 503.
 - **Einstein Trust Layer masks URLs** in agent replies — deliver files to Slack directly instead of via a link.
-

Roadmap

- Finish scan delivery (post image directly into Slack via bot token).
 - ITSM **Case** lifecycle (create/resolve a Case per hardware report).
 - **Smart plug** for a real `restart` power-cycle.
 - Slack-attached-file printing (fetch private Slack files with the bot token).
 - Leave **approval** workflow (manager approve/reject).
 - `logTimesheetBatch` action for reliable multi-day logging.
-

Built iteratively with Claude. Repo: `github.com/uday210/timesheetManagement` · Live: `timesheetmanagement-production.up.railway.app` .